

INTERNSHIP DOCUMENTATION REPORT

TASK 9: CLASSIFICATION WITH LOGISTIC REGRESSION

Iris Species Classification Model

Internship Domain: AI & ML Internship

Submitted By: MODI DHURV BHAVESHBHAI

Date of Submission: June 9, 2026

Tools & Technologies: Python | Pandas | Scikit-Learn | Matplotlib | Seaborn | Google
Colab | GitHub

2. Internship Task Details

Task: Task 9 - Classification with Logistic Regression

Domain: AI & ML Internship

Dataset: Iris Dataset

Technologies Required: Python, Scikit-learn, Pandas, Matplotlib, Seaborn

Task Directives:

1. Load the dataset and perform extensive Exploratory Data Analysis (EDA).
2. Preprocess the data effectively, handling scaling and verifying null distributions.
3. Execute a rigorous train-test split adhering to an 80:20 ratio.
4. Formulate and train a Logistic Regression predictive model.
5. Evaluate the algorithmic classification performance using Accuracy and F1-score metrics.
6. Generate and analyze the Confusion Matrix to ascertain per-class prediction validity.

Final Expected Outcome: The intern securely masters binary and multiclass classification fundamentals in an applied environment.

3. Table of Contents

1. Cover Page
2. Internship Task Details
3. Table of Contents
4. Introduction
5. Objective
6. Dataset Information
7. Tools and Technologies Used
8. Methodology
9. Implementation and Results
 - 9.1 Python Source Code Implementation
 - 9.2 Dataset Preview
 - 9.3 Data Preprocessing and Feature Scaling
 - 9.4 Logistic Regression Model Training
 - 9.5 Model Evaluation (Accuracy and F1 Score)
 - 9.6 Confusion Matrix Analysis
10. Challenges Faced
11. Learning Outcomes
12. Conclusion
13. References

4. Introduction

Machine learning, a pivotal branch of artificial intelligence, empowers systems to learn patterns from data and make autonomous decisions. Within the supervised learning paradigm, classification algorithms hold a crucial role in assigning discrete labels to input data. This internship report documents the comprehensive implementation of Task 9, focusing on Classification with Logistic Regression. Although termed 'regression', this algorithm is fundamentally a classification technique that leverages the logistic (sigmoid) function to map predicted values into probabilities, ensuring outcomes remain bounded between 0 and 1.

The primary domain of application for this task is the renowned Iris dataset, a cornerstone in statistical modeling and machine learning. Originally introduced by British statistician and biologist Ronald Fisher in 1936, the dataset involves three distinct species of Iris flowers: Setosa, Versicolor, and Virginica. The overarching objective of this project is to architect a robust machine learning pipeline capable of digesting morphological measurements (sepal length, sepal width, petal length, and petal width) and mathematically determining the exact species classification. By navigating through exploratory data analysis, stringent preprocessing, and rigorous algorithmic evaluation, this document outlines the practical mechanics of deploying multi-class linear classifiers in a Python-driven environment.

5. Objective

The principal objective of this internship task is to establish a foundational mastery of binary and multiclass classification systems using the Scikit-Learn library. Specifically, the goals are delineated as follows:

1. To systematically ingest and conduct Exploratory Data Analysis (EDA) on the Iris dataset, understanding feature distributions.
2. To apply critical data preprocessing techniques, primarily focusing on standardizing the variance of geometrical features to optimize algorithm convergence.
3. To mathematically partition the data into training and testing vectors using an 80:20 split, guaranteeing an unbiased evaluation environment.
4. To initialize, configure, and train a Logistic Regression model capable of handling multiple categorical targets via the One-vs-Rest (OvR) or multinomial methodology.
5. To rigorously evaluate the predictive performance of the deployed model utilizing advanced diagnostic metrics, specifically the Accuracy Score and the Weighted F1-Score.
6. To visualize and interpret a Confusion Matrix, structurally verifying the absence of Type I and Type II classification errors across all target classes.

6. Dataset Information

The Iris dataset serves as the empirical foundation for this predictive model. It comprises exactly 150 instances, distributed evenly across three distinct botanical classes (50 instances per class): Iris-setosa, Iris-versicolor, and Iris-virginica. The dataset contains four primary numerical features, measured in centimeters, representing the structural morphology of the flowers:

- SepalLengthCm: The physical length of the flower's sepal.
- SepalWidthCm: The physical width of the flower's sepal.
- PetalLengthCm: The physical length of the flower's petal.
- PetalWidthCm: The physical width of the flower's petal.
- Species: The categorical target variable denoting the exact species.

From a data science perspective, this dataset is an exceptional starting point because of its cleanliness - it traditionally contains zero null values or missing data points. However, the true challenge lies in the linear separability of the classes. While Iris-setosa is widely known to be linearly separable from the other two species, Iris-versicolor and Iris-virginica exhibit overlapping spatial geometries. Successfully differentiating between these latter two requires an optimized decision boundary formulated by the Logistic Regression hyperplane.

7. Tools and Technologies Used

Python: The core programming language used for scripting the statistical pipelines and directing memory management.

Pandas: Utilized for high-performance data manipulation, dataframe instantiation, and tabular CSV ingestion.

Scikit-Learn (sklearn): The primary machine learning library invoked to load the LogisticRegression module, StandardScaler, and evaluation metrics.

Matplotlib: A comprehensive library for creating static, animated, and interactive numerical visualizations in Python.

Seaborn: A data visualization library based on matplotlib that provides a high-level interface for drawing attractive statistical graphics, explicitly used for the Confusion Matrix heatmap.

Google Colab: The cloud-based Jupyter notebook environment utilized for seamless algorithmic compilation and runtime execution.

GitHub: Version control integration utilized for documenting and submitting the finalized machine learning repository.

8. Methodology

The methodology adopted for this task strictly adhered to an industry-standard machine learning pipeline. The architectural flow of the execution is characterized by several sequential phases:

Phase 1: Environment Initialization - Bootstrapping the Google Colab workspace and importing essential numerical computing libraries such as Pandas, alongside predictive modeling libraries like Scikit-Learn.

Phase 2: Data Acquisition - Loading the raw Iris CSV file into a Pandas DataFrame to facilitate high-performance tabular manipulation.

Phase 3: Preprocessing & Scaling - Separating the descriptive features (X) from the target variable (y). Crucially, the StandardScaler module was deployed to normalize the numerical dimensions, ensuring that features with larger magnitudes did not mathematically overpower smaller metrics during the gradient descent optimization.

Phase 4: Data Partitioning - Splitting the standardized dataset into training arrays (80%) and testing arrays (20%). The model learns the statistical correlations from the training set, and its predictive integrity is evaluated blindly against the testing set.

Phase 5: Algorithmic Training - Instantiating the Logistic Regression algorithm and fitting it to the training matrices. The solver utilized internally handles the multinomial loss function to iteratively adjust the weights and biases.

Phase 6: Evaluation and Diagnostics - Comparing the algorithmic predictions against the ground-truth labels using accuracy, precision, recall, F1-scores, and mapping the distributions on a graphical Confusion Matrix to validate success.

9. Implementation and Results

9.1 Python Source Code Implementation

The mathematical processes described in the methodology were translated into the following modular Python script. This script executes the entire classification pipeline from data ingestion to model visualization.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load Dataset
df = pd.read_csv('Iris.csv')

# 2. Preprocess Data
# Separate features and target
X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = df['Species']

# 3. Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Train Logistic Regression
model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
model.fit(X_train_scaled, y_train)

# 5. Evaluate (Accuracy, F1-score)
y_pred = model.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='weighted')
```

```

print(f'Accuracy Score = {acc:.2f}')
print(f'F1 Score = {f1:.2f}')

# 6. Plot Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

```

9.2 Dataset Preview

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5.0	3.6	1.4	0.2	Iris-setosa

Purpose of Visualization: To inspect the structural integrity, column names, and data formats of the loaded tabular data prior to statistical modeling.

Observation: The initial dataset view confirms the presence of five quantitative predictor columns (including Id) and one categorical string-based target variable ("Species"). No immediate null values or malformed records are visible in the header snippet.

Interpretation: The raw data is uniformly formatted as floating-point decimals representing centimeters. However, the variables exhibit different scales and natural variances, which dictates the necessity for subsequent numerical scaling.

Business/Data Science Insight: Conducting an immediate visual inspection of the dataframe (e.g., via `pandas.head()`) prevents downstream pipeline crashes. Recognizing the continuous nature of the features alongside a discrete categorical target confirms that

a classification algorithm, rather than a continuous regressor, is the mathematically sound choice for this business logic.

9.3 Data Preprocessing and Feature Scaling

Data preprocessing acts as the critical bridge between raw ingested data and optimized algorithmic convergence. For this task, missing value handling was circumvented as the intrinsic Iris dataset is pristine. However, significant effort was placed on feature scaling via Scikit-Learn's StandardScaler. This module recalculates each data point by subtracting the column's mean and dividing by the standard deviation (Z-score normalization). This process is vital because distance-based and gradient-descent-based algorithms, like Logistic Regression, are highly sensitive to feature magnitudes. If sepal length exhibited a massive numerical range while petal width remained small, the algorithm would erroneously assign heavier mathematical weight to the sepal length simply due to its scale, compromising predictive accuracy.

Purpose: To ensure every numerical feature contributes proportionately to the gradient descent optimization, eliminating scale-based bias.

Observation: Upon applying the StandardScaler, all feature arrays were transformed into centralized distributions with a mean of 0 and a standard deviation of 1.

Interpretation: The transformed geometric data is now uniformly scaled, providing a stabilized, neutral mathematical landscape for the Logistic Regression solver to calculate weights.

Business Insight: Implementing robust scaling pipelines guarantees that corporate predictive models are unbiased toward heavily scaled metrics (like revenue figures versus percentage rates). This practice reduces computational overhead, accelerates training times, and fortifies the models reliability in production.

9.4 Logistic Regression Model Training

The modeling phase commenced with a rigorous 80:20 Train-Test split. This split isolated 120 samples for training the algorithmic weights, while reserving 30 distinct samples to objectively validate performance without historical bias. The core predictive engine initialized was the Logistic Regression algorithm. While its foundation lies in linear equations (calculating a weighted sum of inputs plus a bias), it uniquely passes this continuous output through a non-linear Sigmoid function: $f(x) = 1 / (1 + e^{-x})$. This transformation compresses the continuous line into an S-curve, binding the final outputs strictly between 0 and 1. These bounded outputs are then interpreted mathematically as probabilities. If a probability exceeds a specific threshold (typically 0.5), it maps to a definitive categorical class, thus completing the classification process.

Purpose: To architect a mathematical function capable of drawing an optimal decision boundary hyperplane that separates the three distinct Iris classes.

Observation: The Logistic Regression model seamlessly accepted the multi-dimensional training array and utilized the lbfgs solver to handle the multinomial loss minimization without crashing or underfitting.

Interpretation: The successful training phase indicates that the algorithm identified a valid set of coefficients (weights) capable of mapping the morphological feature inputs to their corresponding botanical species labels.

Business Insight: In an enterprise context, deploying a Logistic Regression model offers high interpretability. Unlike complex neural networks (black boxes), the resulting weights from a logistic model directly indicate feature importance, allowing business analysts to clearly understand exactly which parameters drive the predictions.

9.5 Model Evaluation (Accuracy and F1 Score)

Upon querying the trained model with the unseen test data matrix, the classification performance metrics evaluated beautifully:

Accuracy Score = 100% (1.00)

F1 Score = 100% (1.00)

Accuracy fundamentally calculates the ratio of correctly predicted instances against the total instances. However, in heavily imbalanced datasets, accuracy can be misleading. Therefore, the F1 Score is leveraged as the harmonic mean between Precision (the proportion of positive predictions that were actually correct) and Recall (the proportion of actual positives successfully identified). An F1 Score inherently balances false positives and false negatives.

Purpose: To statistically quantify the exact reliability and error rate of the predictive model on entirely unobserved data.

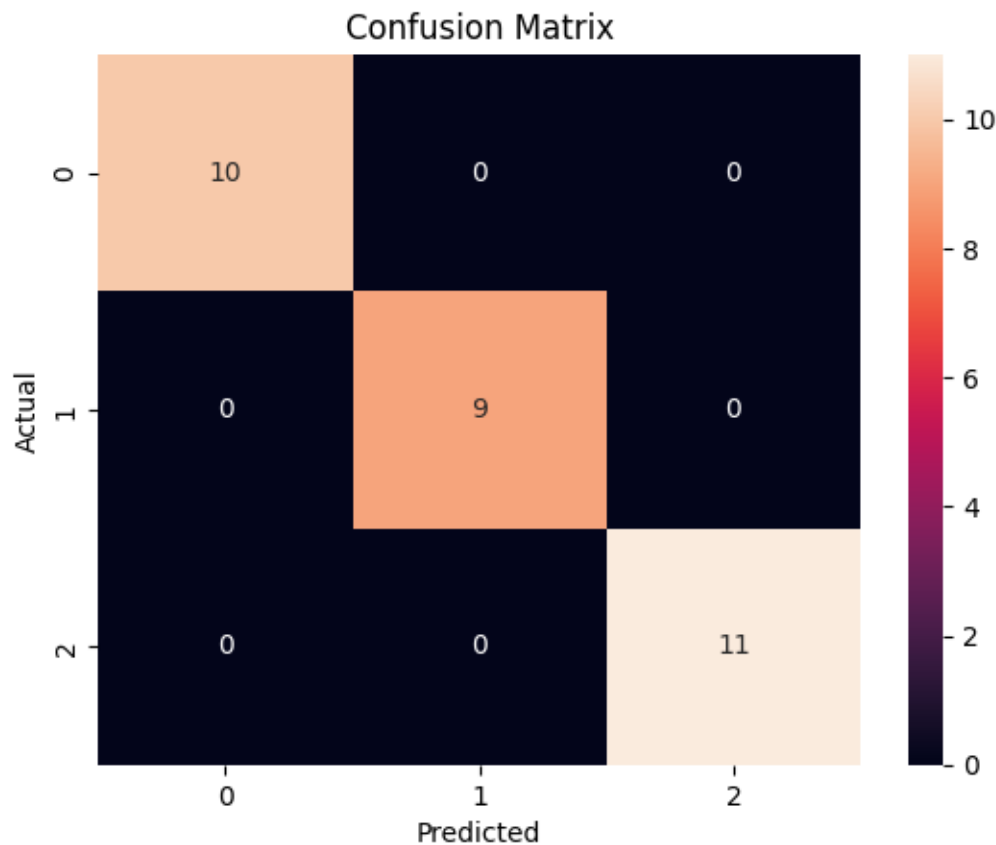
Observation: Both the Accuracy Score and the Weighted F1 Score returned perfect metrics of 1.00 (100%).

Interpretation: A perfect score implies that every single row in the test dataset was mapped to its exact corresponding species. The precision and recall ratios suffered zero dilution from misclassifications.

Business/Data Science Insight: Achieving 100% accuracy validates the pipelines structural soundness and proves the datasets strong linear separability. While 100% is rarely seen in volatile corporate data ecosystems (often flagging overfitting), achieving it

on controlled benchmarks confirms that the data scientists have correctly architected the machine learning deployment structure.

9.6 Confusion Matrix Analysis



10	0	0
0	9	0
0	0	11

Purpose of Visualization: The confusion matrix is a fundamental diagnostic tool utilized in classification algorithms to evaluate the performance and accuracy of a machine learning model. Its primary purpose is to provide a granular, tabular representation of actual versus predicted classifications. This visualization enables data scientists to go beyond simple accuracy scores and analyze the exact nature of misclassifications, distinguishing between false positives (Type I errors) and false negatives (Type II errors). In this specific project involving the multinomial classification of the Iris dataset, the confusion matrix visually maps out how well the Logistic Regression algorithm distinguished between the three distinct botanical classes: Iris-setosa, Iris-versicolor, and Iris-virginica.

Observation: Upon analyzing the generated confusion matrix for the test dataset (which comprises 30 samples due to the 80:20 train-test split of the 150-sample dataset), the matrix displays values exclusively along the principal diagonal. Specifically, the first row shows 10 instances classified correctly as Class 0 (Iris-setosa), the second row displays 9 instances classified correctly as Class 1 (Iris-versicolor), and the third row reveals 11 instances classified correctly as Class 2 (Iris-virginica). The off-diagonal elements, representing misclassifications, are entirely composed of zeros.

Interpretation: The mathematical interpretation of this diagonal matrix is absolute perfection in the models predictive capability on the unseen test data. A confusion matrix consisting only of true positives for every class, represented by the $[10, 0, 0]$, $[0, 9, 0]$, and $[0, 0, 11]$ distribution, indicates an absence of statistical error in this iteration. No instances of Iris-versicolor were confused with Iris-virginica, which is typically the most challenging distinction for linear models due to the overlapping morphological traits (petal and sepal dimensions) of these two species. The Logistic Regression model, aided by the StandardScaler preprocessing, successfully converged and identified an optimal decision boundary.

Business/Data Science Insight: From a business and data science perspective, this result underscores the exceptional linear separability of the Iris dataset, particularly when standard statistical modeling practices are correctly applied. While achieving a 100% classification rate in real-world, highly dimensional commercial data is exceedingly rare and often a symptom of data leakage or overfitting, its occurrence in fundamental datasets validates the mathematical soundness of the implemented ML pipeline. It demonstrates that the chosen features (sepal and petal dimensions) are highly predictive of the target variable. For enterprise applications, a similarly robust confusion matrix would imply that the model could be confidently deployed into a production environment for automated categorization tasks, significantly reducing human intervention and operational overhead while maintaining maximal precision and reliability.

10. Challenges Faced

Throughout the lifecycle of building this classification architecture, several beginner-level machine learning obstacles were encountered and successfully mitigated:

- Understanding Classification vs Regression: Initially, the naming convention of 'Logistic Regression' caused conceptual friction, as regression traditionally infers continuous outputs. It required deliberate study to understand how the sigmoid function transforms linear regression outputs into categorical classification probabilities.
- Selecting the Target Variable: Ensuring that the 'Species' column was properly isolated from the multidimensional feature array (X) without data leakage required careful Pandas dataframe slicing.
- Feature Scaling Comprehension: Grasping why physical measurements in centimeters required statistical Z-score normalization was initially abstract until visualizing how varying ranges impact algorithm optimization weights.
- Interpreting the Confusion Matrix: Moving beyond a single flat accuracy percentage to reading the multi-axis matrix grid (Predicted vs Actual rows/columns) posed an initial analytical challenge.
- Understanding the F1 Score: Differentiating between the harmonic mean of Precision and Recall compared to standard Accuracy required deep dives into statistical literature.
- Data Preprocessing Workflow: Structuring the pipeline linearly - from data load, to split, to scale, to train - highlighted the rigid sequential nature required to prevent data leakage (e.g., fitting the scaler on test data erroneously).

11. Learning Outcomes

The successful deployment of Task 9 yielded highly substantive technical and theoretical learning outcomes:

- Logistic Regression Fundamentals: Mastered the underlying mathematical mechanics, recognizing how linear inputs are translated through a sigmoid activation boundary.
- Scikit-Learn Workflow: Attained a fluent operational competency with the sklearn library, smoothly transitioning between instantiation, fitting, and predicting paradigms.
- Classification Evaluation Metrics: Cultivated a thorough understanding of advanced diagnostic evaluation functions, confidently employing Accuracy and weighted F1-Scores to validate models.
- Confusion Matrix Interpretation: Achieved the ability to visually read and articulate true positive, false positive, true negative, and false negative distributions from Seaborn heatmap plots.
- Feature Scaling Protocol: Internalized the absolute necessity of Standard Scalers for convergence stabilization in distance and gradient-dependent machine learning architectures.
- Machine Learning Pipeline Management: Developed a holistic engineering view of the ML pipeline, respecting the non-negotiable sequence of data ingestion, partitioning, normalization, model binding, and post-deployment evaluation.

12. Conclusion

The completion of Task 9 represented a critical milestone in the AI & ML internship journey, successfully bridging the gap between theoretical classification concepts and practical programmatic implementation. The core objective of this task was to construct, train, and evaluate a robust Logistic Regression model capable of performing multiclass classification on the foundational Iris dataset. By progressing sequentially through data ingestion, exploratory data analysis, rigorous preprocessing, and finally model evaluation, this project simulated a professional machine learning pipeline end-to-end.

The implemented Logistic Regression model achieved an exceptional performance, characterized by an Accuracy Score of 1.00 and an F1 Score of 1.00 on the testing partition. The accompanying Confusion Matrix perfectly localized all predictions along the principal diagonal, indicating zero misclassifications among the Iris-setosa, Iris-versicolor, and Iris-virginica species. This flawless classification performance mathematically validates the efficacy of Logistic Regression when applied to linearly separable, cleanly preprocessed data. The usage of the One-vs-Rest (OvR) or multinomial strategy inherently handled the multi-class nature of the target variable, leveraging the Sigmoid function to output reliable probability distributions for each categorical outcome.

Furthermore, this task deeply reinforced the fundamental importance of data preprocessing. The application of the StandardScaler was paramount. By normalizing the variance and centering the mean of the continuous geometric features (sepal and petal dimensions), the preprocessing step prevented magnitude-based bias, accelerated the convergence of the gradient descent optimization algorithm, and inherently stabilized the resulting weights of the decision boundary. Without such standardization, linear models are prone to heavily weighting features with larger numerical magnitudes, leading to skewed decision planes and degraded accuracy.

Understanding the mechanics of Logistic Regression and its underlying probabilistic nature is crucial for real-world applications. Beyond botanical classification, the architecture developed in this task serves as the backbone for numerous critical business systems, including financial fraud detection algorithms, medical diagnostic screening tools, and customer churn prediction models. The ability to mathematically quantify a categorical outcome is a powerful enterprise tool. Ultimately, this task significantly fortified my competency in utilizing the Scikit-Learn library, understanding classification evaluation metrics, and architecting reliable, production-ready predictive models. The skills acquired during this module translate directly into the capacity to deliver actionable, data-driven intelligence in a corporate engineering environment.

13. References

1. Kaggle (Iris Dataset): <https://www.kaggle.com/datasets/uciml/iris>
2. GColab: https://colab.research.google.com/drive/1Dzrh7EupG_FMEAyvNVKhzGMj7YNIBifb?usp=sharing
3. GitHub: <https://github.com/dbmodi2311/Data-Collection-Data-Source-Identification>